

## 1 Introduction

### K means clustering

K-means clustering is a widely used method for partitioning a set of points into  $k$  clusters by minimizing the within-cluster sum of squared Euclidean distances. It is used in a wide range of applications, including computer vision, feature engineering, and exploratory data analysis. Given a finite point set  $P \subset \mathbb{R}^d$  with  $|P| = n$ , the  $k$ -means cost of a set of centres  $C \subset \mathbb{R}^d$  with  $|C| = k$  is defined as  $\text{cost}_P(C) = \sum_{p \in P} \min_{c \in C} \|p - c\|^2$ . The optimal  $k$ -means cost on  $P$  is  $\text{OPT}_P = \min_{C \subset \mathbb{R}^d, |C|=k} \text{cost}_P(C)$ . This optimisation problem is NP-hard. In practice, a variety of heuristics and approximation algorithms have been designed to improve both the running time and solution quality. In this report, our goal is to benchmark several such  $k$ -means algorithms on a variety of synthetic datasets and compare their clustering cost and running time. In particular, we follow Awasthi et al. [2] for hardness of approximation, Har-Peled and Mazumdar [5] for coresets constructions, and Arthur and Vassilvitskii [1] for  $k$ -means++ seeding.

## 2 Preliminaries

We begin by summarising known hardness and hardness-of-approximation results for Euclidean  $k$ -means, before turning to practical algorithms and empirical evaluation.

### 2.1 Hardness of Euclidean $k$ -means

In this subsection, we summarise the NP-hardness and hardness of approximation of the Euclidean  $k$ -means problem via a reduction from the gap-VERTEX COVER problem, following Section 5 of [2]. Specifically, it is NP-hard to decide whether a given triangle-free graph  $G = (V, E)$  with  $|V| = n$ ,  $|E| = \Theta(n)$ , and  $k = \Theta(n)$  has a vertex cover of size at most  $k$  (*YES-instance*) or whether every vertex cover has size at least  $(1 + \varepsilon)k$  (*NO-instance*).

We summarise the reduction from this gap-VERTEX COVER problem to the Euclidean  $k$ -means problem as presented in Sections 3 and 4 of the paper.

**Reduction.** Given a triangle-free VERTEX COVER instance  $(G = (V, E), k)$  with  $|V| = n$  and  $|E| = m$ : For each edge  $e = (i, j) \in E$ , create a point  $x_e := e_i + e_j \in \mathbb{R}^n$ , where  $\{e_1, \dots, e_n\}$  are standard basis vectors ( $\|x_e\|^2 = 2$ ). Set the number of centres in the  $k$ -means instance to  $k$ . The objective is the standard squared Euclidean  $k$ -means cost:  $\sum_{e \in E} \min_{c \in C} \|x_e - c\|^2$ , for  $|C| = k$ .

**(1) Completeness.** For a cluster  $F \subseteq E$  with  $m_F = |F|$  and vertex degrees  $d_F(u)$  in  $(V, F)$ , the centroid is  $\mu_F = \frac{1}{m_F} \sum_{e \in F} x_e$ . Then,

$$\text{Cost}(F) = \sum_{e \in F} \|x_e - \mu_F\|^2 = \sum_{e \in F} \|x_e\|^2 - m_F \|\mu_F\|^2 = 2m_F - \frac{1}{m_F} \sum_u d_F(u)^2 = \sum_u d_F(u) \left(1 - \frac{d_F(u)}{m_F}\right).$$

If  $G$  has a vertex cover  $S^* = \{v_1, \dots, v_k\}$ , assign each edge to a chosen endpoint:  $F_v = \{e = (v, w) \in E\}$  for  $v \in S^*$ . Each  $(V, F_v)$  is a star, whose cost by the formula above is  $m_{F_v} - 1$ . Summing  $\sum_{v \in S^*} (m_{F_v} - 1) = m - k$ , hence  $\text{OPT}_{\text{km}} \leq m - k$ .

**(2) Soundness.** Assume a clustering  $\Gamma = \{F_1, \dots, F_k\}$  satisfies  $\sum_{i=1}^k \text{Cost}(F_i) \leq m - (1 - \delta)k$ . For each  $F_i$  let  $m_i := |F_i|$  and define  $\delta_i := \text{Cost}(F_i) - (m_i - 1) \Rightarrow \frac{1}{m_i} \sum_u d_{F_i}(u)^2 = m_i + 1 - \delta_i$ , and note that  $m - (1 - \delta)k \geq \sum_i \text{Cost}(F_i) = \sum_i ((m_i - 1) + \delta_i) = m - k + \sum_i \delta_i \Rightarrow \sum_i \delta_i \leq \delta k$ . Hence at least  $(1 - 2\delta)k$  clusters have  $\delta_i \leq \frac{1}{2}$ .

**Lemma 1.** If  $\frac{1}{m_F} \sum_u d_F(u)^2 = m_F + 1 - \delta_F$  with  $\delta_F \leq \frac{1}{2}$ , then  $F$  is a star or triangle and  $\delta_F = 0$ . (Refer to Appendix A.1 for proof)

Since  $G$  is triangle-free, all such clusters are stars (each center covers its edges). At most  $2\delta k$  clusters may have  $\delta_i > \frac{1}{2}$ ; by Lemma 11 (part 1), selecting two vertices per such cluster covers all but  $\delta_i$  edges in that cluster. Across all clusters, this uses at most  $(1 - 2\delta)k + 2(2\delta k) = (1 + 2\delta)k$  vertices and leaves at most  $\sum_i \delta_i \leq \delta k$  edges uncovered; adding one endpoint per such edge yields a vertex cover of size at most  $(1 + 2\delta)k + \delta k = (1 + 3\delta)k$ . Thus any near-optimal  $k$ -means solution yields a  $(1 + O(\delta))k$  vertex cover.

**Conclusion.** This shows that achieving a  $(1 + \epsilon)$ -approximation for Euclidean  $k$ -means is NP-hard and that no PTAS exists unless  $P = NP$  (see Appendix A.2 for the full argument). In light of these hardness results, we now turn to practical heuristics and approximation schemes that perform well in practice.

## 2.2 Lloyd’s Algorithm and $k$ -means++

Given that exact optimization and even close approximation of  $k$ -means is computationally hard, most practical work relies on heuristic algorithms. We begin by recalling Lloyd’s algorithm and its  $k$ -means++ initialization variant. Detailed pseudocode for Lloyd’s algorithm and  $k$ -means++ seeding is given in Appendix B.1 and Appendix B.2 respectively.

**Lloyd’s algorithm.** [6] Initialize  $k$  centres (e.g., randomly from  $P$ ). Then iterate:

- (i) *Assignment:* Assign each point  $p$  to its nearest centre  $c_i$ ;
- (ii) *Update:* Set each centre  $c_i$  to the mean of its assigned points.
- (iii) Repeat (i) and (ii) until convergence (a local minimum of the objective).

**$k$ -means++ Algorithm** [1] The difference between  $k$ -means++ and Lloyd’s is in the initialisation of centres, since  $k$ -means++ runs Lloyd’s algorithm afterwards. The initialisation is as follows [1]:

Pick the first centre uniformly from  $P$ . For  $\ell = 2, \dots, k$ , pick  $c_\ell = p \in P$  with probability proportional to the squared distance to the current set of centres:  $\mathbb{P}[p] = \frac{D(p)^2}{\sum_{y \in P} D(y)^2}$  and  $D(p) = \min_{c \in \{c_1, \dots, c_{\ell-1}\}} \|p - c\|_2$ . This spreads centres out before running Lloyd’s iterations; in expectation, the seeding achieves an  $O(\log k)$ -approximation to the optimal  $k$ -means cost.

## 2.3 $(1 + \epsilon)$ -approximation using CoreSets

While Lloyd’s algorithm and  $k$ -means++ operate directly on the full dataset, coreset constructions aim to accelerate clustering by compressing the data into a small weighted subset that approximately preserves the  $k$ -means objective.

The computational bottleneck of clustering is its dependence on  $n$ . By replacing  $P$  with a coreset  $S$  of size  $|S| = O(\frac{k}{\epsilon^d} \log n)$ , all subsequent optimization steps depend on  $|S| \ll n$ . Thus, the algorithm achieves *near-linear time* in  $n$ , plus terms depending only on  $k$ ,  $d$ , and  $\epsilon$ .

**Definition 1** ( $(k, \epsilon)$ -Coreset). *According to Section 6 of [5], a subset  $S \subseteq P$  is a  $(k, \epsilon)$ -coreset for  $P$  such that for every set of  $k$  centres  $C$ , we have  $(1 - \epsilon) \text{cost}_P(C) \leq \text{cost}_S(C) \leq (1 + \epsilon) \text{cost}_P(C)$ . The clustering cost of the coreset  $S$  is within a factor  $(1 \pm \epsilon)$  of the cost when evaluated on the full dataset  $P$ .*

### Deriving the $(1 + \epsilon)$ Approximation

**Coreset Guarantee.** From Definition 1, for every  $k$ -centre set  $C$ ,

$$(1 - \epsilon) \text{cost}_P(C) \leq \text{cost}_S(C) \leq (1 + \epsilon) \text{cost}_P(C). \quad (1)$$

Let  $C_P^*$  be the optimal centres on  $P$ , and  $C_S^*$  be the optimal centres on  $S$ . Then, from both inequalities we obtain  $\text{cost}_P(C_S^*) \leq \frac{1}{1 - \epsilon} \text{OPT}_S$  and  $\text{OPT}_S \leq (1 + \epsilon) \text{OPT}_P$ . Combining both inequalities gives us  $\text{cost}_P(C_S^*) \leq \frac{1 + \epsilon}{1 - \epsilon} \text{OPT}_P$ .

**Discrete Centroid Set.** To make computation finite, the algorithm constructs an  $\epsilon$ -approximate centroid set  $U$  for  $S$  such that there exists some  $C_U \subseteq U$ ,  $|C_U| = k$ , with  $\text{cost}_S(C_U) \leq (1 + \epsilon) \text{OPT}_S$ . By enumeration of  $k$ -tuples over  $U$ , the algorithm finds  $\tilde{C}$  satisfying  $\text{cost}_S(\tilde{C}) \leq (1 + \epsilon) \text{OPT}_S$ .

Using the left inequality of Equation (1) to relate to  $P$ ,

$$\text{cost}_P(\tilde{C}) \leq \frac{1}{1 - \epsilon} \text{cost}_S(\tilde{C}) \leq \frac{(1 + \epsilon)^2}{1 - \epsilon} \text{OPT}.$$

**Choosing  $\varepsilon$ .** For small  $\varepsilon$ , the multiplicative factor expands as  $\frac{(1+\varepsilon)^2}{1-\varepsilon} = 1 + 3\varepsilon + 4\varepsilon^2 \leq 1 + 4\varepsilon$  for  $\varepsilon \leq \frac{1}{5}$ . Setting  $\varepsilon' = 4\varepsilon$  ensures that the total factor is at most  $(1 + \varepsilon')$ :  $\text{cost}_P(\tilde{C}) \leq (1 + \varepsilon') \text{OPT}$

### 3 Experimentation

#### 3.1 Algorithms

We empirically compare the following algorithms:

**Bruteforce:** A baseline multi-start implementation that runs Lloyd’s algorithm 1000 times with different random initializations and returns the best solution found. This serves as a heuristic proxy for the global optimum.

**Lloyd’s algorithm:** The standard iterative refinement algorithm that alternates between assigning each point to its nearest centre and recomputing each centre as the mean of its assigned points.

**$k$ -means++ (PP):** Lloyd’s algorithm with  $k$ -means++ initialization, which samples initial centres with probability proportional to their squared distance from previously chosen centres, biasing the initialization toward well-separated centres.

**Coreset:** A method that first builds a coreset  $S$  of the dataset, runs a  $k$ -means routine on  $S$  to obtain an initial set of centres, and finally refines these centres with a few iterations of Lloyd’s algorithm on the full dataset.

Pseudocode for these algorithms are given in Appendix B.

#### 3.2 Experimental Methodology and Datasets

To compare these algorithms, we conducted experiments on six synthetic datasets designed to exhibit different cluster geometries and difficulty levels:

- **Blobs** ( $k = 5$ ): Well-separated isotropic Gaussian clusters in  $\mathbb{R}^{10}$ .
- **Stretched Blobs** ( $k = 5$ ): Anisotropic Gaussian blobs obtained by linearly transforming isotropic blobs, resulting in elongated clusters.
- **Unbalanced Blobs** ( $k = 5$ ): Gaussian clusters with differing standard deviations, giving rise to clusters of varying density and scale.
- **Random** ( $k = 5$ ): A near-uniform baseline in  $\mathbb{R}^{10}$  without any clear cluster structure.
- **Circles** ( $k = 2$ ): Two concentric hollow circles in  $\mathbb{R}^2$ .
- **Moons** ( $k = 2$ ): Two interleaving moon-shaped clusters in  $\mathbb{R}^2$  with curved decision boundaries.

Each dataset contains 10,000 points. For each dataset–algorithm combination, we record:

1. **Inertia:** The within-cluster sum of squared distances (the  $k$ -means objective; lower is better).
2. **Time:** The wall-clock running time in seconds.
3. **Iterations:** The number of Lloyd-style update steps until convergence.

We summarise the performance of all methods on these datasets in Table 1 and discuss the results in the next subsection.

#### 3.3 Results and Discussion

| Dataset            | Inertia    |           |           |           | Time (s)   |         |       |       | Iterations |         |       |    |
|--------------------|------------|-----------|-----------|-----------|------------|---------|-------|-------|------------|---------|-------|----|
|                    | Bruteforce | Coreset   | Lloyd     | PP        | Bruteforce | Coreset | Lloyd | PP    | Bruteforce | Coreset | Lloyd | PP |
| Blobs              | 2.253e+05  | 2.256e+05 | 4.907e+05 | 2.253e+05 | 53.201     | 0.501   | 0.114 | 0.301 | 3          | 2       | 45    | 2  |
| Blobs (Stretched)  | 9.431e+06  | 1.667e+07 | 9.431e+06 | 9.431e+06 | 21.132     | 0.648   | 0.023 | 0.271 | 4          | 4       | 5     | 3  |
| Blobs (Unbalanced) | 7.670e+05  | 1.074e+06 | 8.104e+05 | 7.671e+05 | 109.736    | 2.672   | 0.078 | 0.349 | 52         | 33      | 34    | 46 |
| Circles            | 3.999e+03  | 4.009e+03 | 4.003e+03 | 4.007e+03 | 17.351     | 0.227   | 0.019 | 0.046 | 11         | 13      | 8     | 11 |
| Moons              | 4.138e+03  | 4.138e+03 | 4.138e+03 | 4.138e+03 | 8.862      | 0.181   | 0.021 | 0.056 | 12         | 10      | 13    | 12 |
| Random             | 6.764e+03  | 6.814e+03 | 6.798e+03 | 6.782e+03 | 186.695    | 4.086   | 0.244 | 0.403 | 64         | 54      | 110   | 79 |

Table 1: K-Means Variants Comparison on Different Datasets

**Inertia.** For most datasets, the three full-data methods (Bruteforce, Lloyd,  $k$ -means++) achieve very similar inertia, indicating that Lloyd’s algorithm and  $k$ -means++ typically find solutions close to the best among 1000 random restarts. For example, on stretched blobs all three obtain an inertia of approximately  $9.43 \times 10^6$ . On the low-dimensional circles and moons datasets, all four methods, including the coresets-based approach, achieve inertia around  $4.0 \times 10^3$ , suggesting that the geometry of those datasets is easy for all variants.

The coresets-based method exhibits a small but systematic degradation in solution quality on several datasets. On stretched blobs, its inertia ( $1.67 \times 10^7$ ) is noticeably larger than that of the full-data methods. Similar behaviour appears on blobs, unbalanced blobs, and random, where the coresets inertia is consistently higher. This is expected: the algorithm optimizes an approximate objective on a compressed representation, and aggressive compression can undersample small or diffuse clusters, especially in high or unbalanced dimensions.

**Runtime.** In terms of runtime, bruteforce is by far the slowest algorithm, as it runs Lloyd’s algorithm 1000 times. Lloyd’s algorithm is usually the fastest in wall-clock time, with  $k$ -means++ slightly slower because of the additional initialization overhead. The coresets method lies in between brute-force and full-data Lloyd: it is much faster than bruteforce but consistently slower than Lloyd and  $k$ -means++ on these instance sizes, suggesting that the overhead of constructing the coresets is not amortized at  $n = 10,000$  when efficient vectorised routines are available on the full dataset.

**Iterations.** The iteration counts provide additional insight into convergence behaviour. On datasets with clear low-dimensional structure (blobs, stretched blobs, circles, moons), all methods converge in at most a few tens of iterations, with  $k$ -means++ often requiring fewer iterations than standard Lloyd (for example, three versus five on stretched blobs). This is consistent with the usual empirical claim that  $k$ -means++ gives a better starting point and accelerates convergence. On the more challenging random and unbalanced blobs datasets, the number of iterations grows substantially, especially for Lloyd and  $k$ -means++, but the per-iteration cost remains small in the vectorised implementation, leading to modest total runtimes.

Overall, these results suggest that  $k$ -means++ and Lloyd’s algorithm typically achieve near-optimal solutions on our synthetic datasets, while coresets-based methods incur a modest loss in objective value in exchange for potential gains on truly large-scale datasets, consistent with their theoretical motivation.

## 4 Conclusion

In conclusion, we studied Euclidean  $k$ -means clustering from both a theoretical and empirical perspective. On the complexity side, we outlined a gap-preserving reduction from triangle-free Vertex Cover to Euclidean  $k$ -means, showing that Euclidean  $k$ -means is APX-hard and therefore does not admit a PTAS unless  $P = NP$ . On the algorithmic side, we presented a coresets-based  $(1 + \varepsilon)$ -approximation scheme and explained how its guarantees follow from the coresets property and the construction of an approximate centroid set.

Our experiments compared brute-force (random restarts), Lloyd’s algorithm,  $k$ -means++, and a coresets-based variant on several synthetic datasets. We observed that  $k$ -means++ and Lloyd typically provide inertia values close to the best of 1000 random restarts, while the coresets-based method experiences a small loss in objective value but can still be competitive in settings where constructing the coresets is cheaper than running many restarts. In the regime we tested, the overhead of coresets construction dominated, but for larger datasets or streaming data, coresets are expected to offer more substantial benefits. This is especially so in the larger practical datasets, where the number of data points are in the millions.

While our report has focused primarily on coresets-based approximations and classical Lloyd-style algorithms, a complementary line of research aims to scale  $k$ -means to web-scale datasets and highly parallel architectures. The standard  $k$ -means++ initialization algorithm [1] is inherently sequential and therefore becomes a bottleneck at web scale. To address this, scalable variants such as Scalable  $k$ -means++ [4] have been proposed, along with alternative seeding schemes, including Markov-chain-based methods [3]. Conceptually, scalable seeding improves initialization quality and parallel efficiency, whereas coresets reduces the effective problem size. Thus, combining coresets constructions with scalable seeding strategies is a promising avenue for achieving both statistical robustness and computational efficiency on truly large-scale clustering problems.

## References

- [1] David Arthur and Sergei Vassilvitskii. *k-means++: The Advantages of Careful Seeding*. Technical Report 2006-13. Stanford InfoLab, June 2006. URL: <http://ilpubs.stanford.edu:8090/778/>.
- [2] Pranjal Awasthi et al. “The Hardness of Approximation of Euclidean k-means”. In: *CoRR* abs/1502.03316 (2015). arXiv: 1502.03316. URL: <http://arxiv.org/abs/1502.03316>.
- [3] Olivier Bachem et al. “Fast and provably good seedings for k-means”. In: *Advances in neural information processing systems* 29 (2016).
- [4] Bahman Bahmani et al. “Scalable k-means++”. In: *Proc. VLDB Endow.* 5.7 (Mar. 2012), pp. 622–633. ISSN: 2150-8097. DOI: 10.14778/2180912.2180915. URL: <https://doi.org/10.14778/2180912.2180915>.
- [5] Sariel Har-Peled and Soham Mazumdar. “Coresets for  $k$ -Means and  $k$ -Median Clustering and their Applications”. In: *CoRR* abs/1810.12826 (2018). arXiv: 1810.12826. URL: <http://arxiv.org/abs/1810.12826>.
- [6] S. Lloyd. “Least squares quantization in PCM”. In: *IEEE Transactions on Information Theory* 28.2 (1982), pp. 129–137. DOI: 10.1109/TIT.1982.1056489.

## A Proofs

### A.1 Proof for Lemma 1

**Lemma 1.** Let  $G = (V, E)$  be a simple graph with  $m = |E| \geq 1$  and degree sequence  $(d_1, \dots, d_n)$ . Define  $\delta \geq 0$  by

$$\frac{1}{m} \sum_{u \in V} d_u^2 = m + 1 - \delta.$$

Then:

1. (Heavy edge) There exists an edge  $\{u, v\} \in E$  such that

$$d_u + d_v - 1 \geq m - \delta.$$

2. (Structure for small  $\delta$ ) If  $\delta < \frac{1}{2}$ , then  $\delta = 0$  and  $G$  is either a star (for any  $m$ ) or a triangle (when  $m = 3$ ).

*Proof. Part 1.* Use the identity  $\sum_{u \in V} d_u^2 = \sum_{\{x, y\} \in E} (d_x + d_y)$ . gives

$$\frac{1}{m} \sum_{\{x, y\} \in E} (d_x + d_y) = \frac{1}{m} \sum_{u \in V} d_u^2 = m + 1 - \delta.$$

Hence the average of  $d_x + d_y$  over a uniformly random edge is  $m + 1 - \delta$ , so there exists  $\{u, v\} \in E$  with  $d_u + d_v \geq m + 1 - \delta$ , i.e.  $d_u + d_v - 1 \geq m - \delta$ .

**Part 2.** Assume  $\delta < \frac{1}{2}$ . Let  $\{u, v\}$  be an edge maximizing  $d_u + d_v$ , and relabel so that  $\alpha := d_u \geq \beta := d_v$ . From Part 1,  $d_u + d_v - 1 \geq m - \delta$ .

*Case A:*  $\beta \leq 1$ . Then  $d_u + d_v - 1 \geq m - \delta > m - \frac{1}{2}$ , so  $d_u + d_v - 1 \geq m$ ; hence every edge is incident to  $u$  or  $v$ . If  $\beta = 1$ , all edges are incident to  $u$  and  $G$  is a star (center  $u$ , degree  $m$ ). For a star,  $\sum d_u^2 = m^2 + m = m(m + 1)$ , so  $\delta = 0$ . (The option  $\beta = 0$  is impossible since  $\{u, v\} \in E$ .)

*Case B:*  $\beta \geq 2$ . Consider the degree sequence  $\mathbf{d} = (d_1, \dots, d_n)$  (sorted nonincreasingly) and define

$$\mathbf{d}' = \left( \alpha, \beta, \underbrace{2, \dots, 2}_{\beta-1 \text{ times}}, \underbrace{1, \dots, 1}_{\alpha-\beta \text{ times}} \right).$$

$\mathbf{d}$  is majorized by  $\mathbf{d}'$ . Since  $x \mapsto x^2$  is convex,  $\sum d_u^2$  is Schur-convex, so

$$m(m + 1 - \delta) = \sum_u d_u^2 \leq \sum_u (d'_u)^2 = \alpha^2 + \beta^2 + 4(\beta - 1) + (\alpha - \beta).$$

Rearranging yields

$$0 \leq (\alpha + \beta - 1)\delta + 2\alpha(1 - \beta) + 4(\beta - 1).$$

Since  $\beta \geq 2$  and  $\alpha \geq \beta$ ,

$$2\alpha(1 - \beta) \leq 0 \quad \text{and} \quad 4(\beta - 1) \geq 4,$$

so

$$2\alpha \leq \frac{\alpha + \beta - 1}{\beta - 1} \delta + 4 \leq \alpha \delta + 4 + \delta,$$

where we used  $\frac{\alpha + \beta - 1}{\beta - 1} \leq \alpha + 1$  for  $\alpha \geq \beta \geq 2$ . Hence

$$(2 - \delta)\alpha \leq 4 + \delta.$$

With  $\delta < \frac{1}{2}$ , we get  $\alpha < 3$ , so  $\alpha \leq 2$ . Since  $\alpha \geq \beta \geq 2$ , we conclude  $\alpha = \beta = 2$ .

Now  $\alpha + \beta - 1 = 3$  equals the number of edges incident to the pair  $\{u, v\}$ , hence  $m \leq 3$ . Because  $\beta \geq 2$ , necessarily  $m = 3$ . The only simple connected graphs with  $m = 3$  and maximum degree 2 are the triangle  $C_3$  and the 4-path  $P_4$ ; a direct computation shows

$$\delta(C_3) = 0, \quad \delta(P_4) = 4 - \frac{1}{3}(2^2 + 2^2 + 1^2 + 1^2) = \frac{2}{3} > \frac{1}{2}.$$

Thus  $\delta < \frac{1}{2}$  forces the triangle, and in either Case A or B we have  $\delta = 0$ .

Therefore, if  $\delta < \frac{1}{2}$ , then  $\delta = 0$  and  $G$  is either a star (any  $m$ ) or a triangle ( $m = 3$ ), as claimed.  $\square$

## A.2 Hardness of approximation

Part (1) gives  $\text{OPT}_{\text{km}} \leq m - k$  for a VERTEX COVER of size  $k$ , and Part (2) converts any clustering of cost  $\leq m - (1 - \delta)k$  into a  $(1 + O(\delta))k$  cover. Section 5 provides a constant-gap triangle-free VERTEX COVER instance with  $m = \Theta(n)$  and  $k = \Theta(n)$ . Under this reduction, YES-instances satisfy  $\text{OPT}_{\text{km}} \leq m - k$ , while NO-instances satisfy  $\text{OPT}_{\text{km}} \geq m - (1 - c\varepsilon)k = (m - k) + c\varepsilon k$  for some constant  $c > 0$ . Therefore,  $\frac{\text{OPT}_{\text{km}}(\text{NO})}{\text{OPT}_{\text{km}}(\text{YES})} \geq 1 + \Omega(\varepsilon)$ , since  $m - k = \Theta(m)$  and  $k = \Theta(m)$ . Any  $(1 + \delta)$ -approximation with  $\delta < \alpha\varepsilon$  (for a constant  $\alpha > 0$ ) would separate the two cases in polynomial time, contradicting the NP-hardness of the source gap. Hence Euclidean  $k$ -means admits no PTAS (unless  $P = NP$ ) and is **APX-hard**.

## B Algorithmic Pseudocode

### B.1 Lloyd's Algorithm

---

**Algorithm 1** Lloyd's Algorithm for  $k$ -Means

---

- 1: **Input:** Dataset  $P \subset \mathbb{R}^d$ , number of clusters  $k$
  - 2: Initialize centres  $C^{(0)} = \{c_1^{(0)}, \dots, c_k^{(0)}\}$  (for example, random points from  $P$ )
  - 3: **for**  $t = 0, 1, 2, \dots$  until convergence **do**
  - 4:     Assign each  $p \in P$  to its nearest centre in  $C^{(t)}$  to form clusters  $P_1^{(t)}, \dots, P_k^{(t)}$
  - 5:     **for**  $i = 1$  to  $k$  **do**
  - 6:         Update  $c_i^{(t+1)} \leftarrow \frac{1}{|P_i^{(t)}|} \sum_{p \in P_i^{(t)}} p$
  - 7:     **if**  $C^{(t+1)} = C^{(t)}$  **then**
  - 8:         **break**
  - 9: **return** final centres  $C^{(t)}$  and clusters
- 

### B.2 $k$ -Means++ Initialization

---

**Algorithm 2**  $k$ -Means++ Initialization

---

- 1: **Input:** Dataset  $P \subset \mathbb{R}^d$ , number of clusters  $k$
  - 2: Choose  $c_1$  uniformly at random from  $P$
  - 3: **for**  $\ell = 2$  to  $k$  **do**
  - 4:     For each  $p \in P$ , compute  $D(p)^2 = \min_{1 \leq j < \ell} \|p - c_j\|_2^2$
  - 5:     Choose  $c_\ell$  from  $P$  with probability proportional to  $D(p)^2$
  - 6: **return** initial centres  $\{c_1, \dots, c_k\}$
- 

### B.3 Coreset-Based $k$ -Means

---

**Algorithm 3** Coreset-Based k-Means (Sensitivity Sampling)

---

- 1: **Input:** Dataset  $P \subset \mathbb{R}^d$ , number of clusters  $k$ , approximation  $\epsilon$
  - 2:  $C_{init} \leftarrow$  Run k-Means++ on  $P$  to get  $k$  initial centers
  - 3: **for** each point  $p \in P$  **do**
  - 4:      $d(p)^2 \leftarrow \min_{c \in C_{init}} \|p - c\|_2^2$
  - 5:  $\Phi \leftarrow \sum_{p \in P} d(p)^2$  ▷ Total cost of initial clustering
  - 6: **for** each point  $p \in P$  **do**
  - 7:      $\mathbb{P}(p) \leftarrow \frac{d(p)^2}{\Phi} + \frac{1}{2|P|}$  ▷ Calculate sensitivity-based probability
  - 8:  $m \leftarrow \mathcal{O}(k \log |P| / \epsilon^2)$  ▷ Set coreset size
  - 9:  $S \leftarrow$  Sample  $m$  points from  $P$  with replacement, using probabilities  $\mathbb{P}(p)$
  - 10: Initialize weights array  $W$
  - 11: **for** each  $s_i \in S$  **do**
  - 12:      $W[i] \leftarrow 1/(m \cdot \mathbb{P}(s_i))$  ▷ Calculate importance weight
  - 13:  $\tilde{C} \leftarrow$  Run *weighted* k-Means on the coreset  $(S, W)$  ▷ Update step becomes:  $c_j \leftarrow \frac{\sum_{s_i \in S_j} w_i \cdot s_i}{\sum_{s_i \in S_j} w_i}$
  - 14:  $L \leftarrow$  Assign each point  $p \in P$  to its nearest centroid in  $\tilde{C}$
  - 15:
  - 16: **Return:** Centroids  $\tilde{C}$  and labels  $L$
-